HB++ Implementation Plan: Cross-Platform Google Authentication & RBAC

Goal Description

Build the **HB++ Domain-Restricted Google Authentication Application** as specified in `PRD.md` and `ARCHITECTURE.md`. The product is an enterprise-grade cross-platform application (Web, Android, iOS) built using a unified Expo / React Native codebase. It enforces Google OAuth 2.0 authentication strictly restricted to the official email domain `@hbplus.fit`, provides session persistence, protected navigation, user profiles, and full Role-Based Access Control (Admin and User tiers with an Admin User Management Dashboard). All bonus features in the PRD (Firestore user profile management, Admin/User RBAC, and Admin management dashboard) are treated as mandatory requirements.

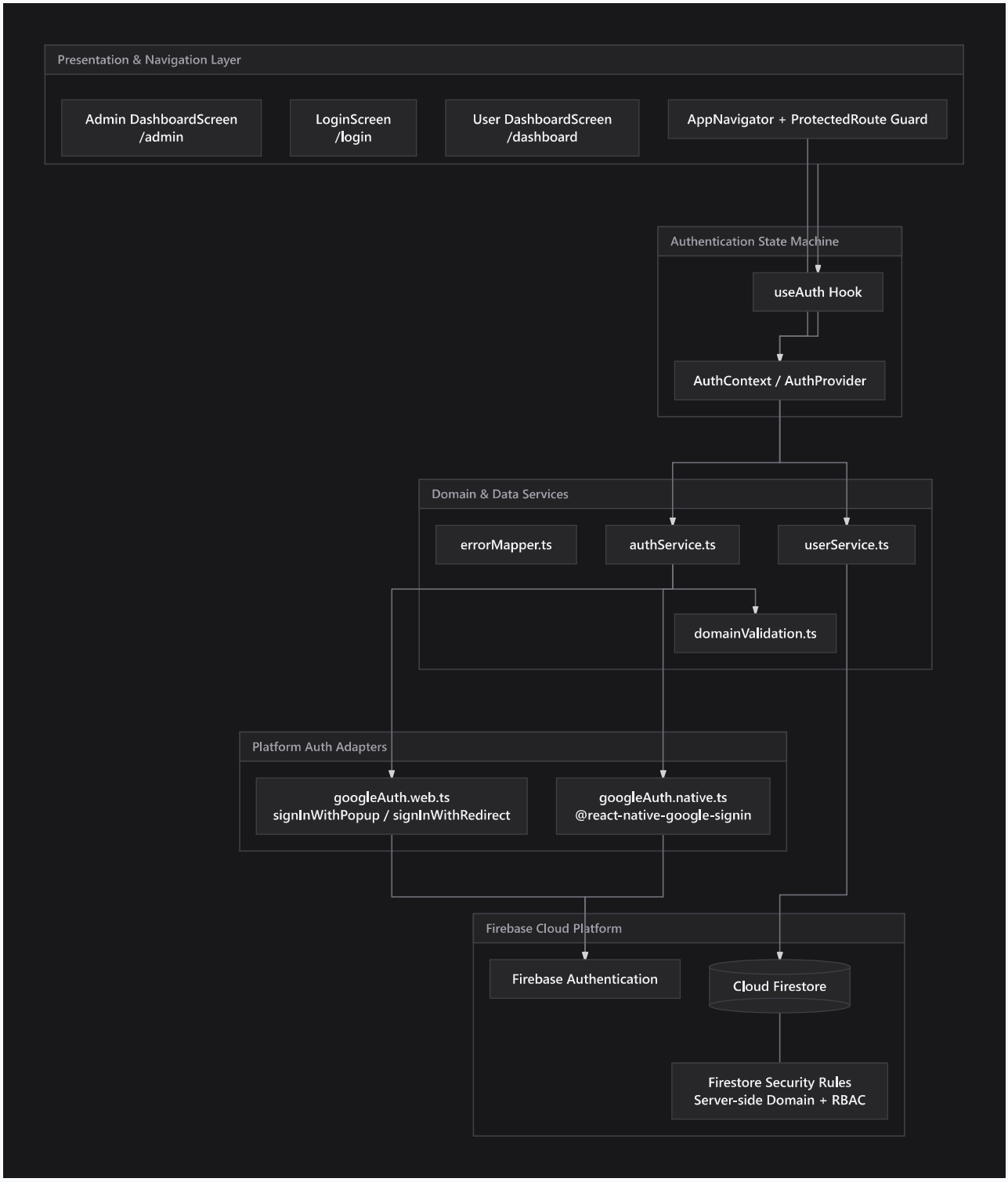
User Review Required

> [!IMPORTANT]
> **Greenfield Repository Status**: The current workspace at `C:\Users\chlok\Downloads\HB++` contains only `PRD.md` and `ARCHITECTURE.md`. There is no pre-existing code, `package.json`, or `.git` repository. Implementation will bootstrap an Expo TypeScript project directly in this directory while strictly preserving `PRD.md` and `ARCHITECTURE.md`.

> [!WARNING]
> **Admin Role Assignment & Bootstrapping**: Standard users must never be able to elevate their own privileges. To satisfy mandatory RBAC requirements without requiring manual Firestore console edits:
>
> 1. An initial bootstrap list or designated email (e.g. `admin@hbplus.fit` or specified via `EXPO_PUBLIC_INITIAL_ADMIN_EMAIL`) will receive the `admin` role upon first login.
> 2. Firestore Security Rules will strictly enforce that only authenticated users whose Firestore profile already has `role == 'admin'` can alter other users' roles or list all user documents.
> 3. Standard users can only read and update their own non-role profile attributes (`displayName`, `photoURL`, `lastLoginAt`).

> NOTE
>
> **Native Mobile vs Web OAuth Architecture**:
>
> - **Web**: Uses standard Firebase client SDK `signInWithPopup(auth, googleProvider)` with automatic fallback to `signInWithRedirect` when popups are blocked by browsers.
> - **Android / iOS**: Native Google Sign-In requires an Expo development build (`expo run:android` / `expo run:ios` / EAS Build) using `@react-native-google-signin/google-signin` and credential exchange `GoogleAuthProvider.credential(idToken)` . Web development and Netlify deployment proceed without requiring native build tools on local machines.

Proposed System Architecture & Flow



Detailed Phased Implementation Plan

Phase 1: Repository Analysis, Foundation & Expo Setup

- **Objective**: Establish the workspace foundation, initialize Expo SDK with TypeScript, configure aliases, theming, and build tooling.
- **Specific Tasks**:
    1. Initialize Git repository (`git init`) and commit documentation baseline.
    2. Create standard Expo app configuration using TypeScript (`npx create-expo-app@latest . --template blank-typescript` or modular scaffolding to prevent overwriting existing Markdown docs).
    3. Install core dependencies: `expo`, `react-native`, `react-native-web`, `react-dom`, `@expo/metro-runtime`.
    4. Configure `app.json` with app name (`HB Plus`), package identifier (`com.hbplus.auth`), iOS bundle identifier (`com.hbplus.auth`), and web bundler settings (`metro`).
    5. Setup `tsconfig.json` with strict type checking and path aliases (`@components/*`, `@screens/*`, `@services/*`, etc.).
    6. Create directory structure matching ARCHITECTURE Section 5.
    7. Implement `src/styles/theme.ts` with color palette, typography, touch target constants (min 44px), spacing, and responsive breakpoints.
    8. Create `.env.example` and `.gitignore` ensuring environment secrets are never committed.
- **Relevant Files**:
    - `package.json`, `app.json`, `tsconfig.json`, `metro.config.js`, `.gitignore`, `.env.example`
    - `src/styles/theme.ts`
- **Prerequisites**: Node.js v22+, npm 11+.
- **Security Considerations**: Strict `.gitignore` preventing `.env`, `.env.local`, and credential files from being tracked.
- **Testing & Validation**: Run `npx expo start --web` to confirm blank app boots in the browser.
- **Deliverables**: Working Expo project running on web with verified TypeScript compilation.

Phase 2: Firebase SDK & Multi-Platform Google OAuth Configuration

- **Objective**: Configure Firebase client SDK with platform-appropriate session persistence and establish web/native Google authentication adapters.

- **Specific Tasks**:
    1. Install Firebase v10+ client library (`firebase`), `@react-native-async-storage/async-storage` .
    2. Implement `src/constants/config.ts` to validate and expose `EXPO_PUBLIC_FIREBASE_*` and `EXPO_PUBLIC_ALLOWED_EMAIL_DOMAIN` environment variables with clear startup assertions.
    3. Implement `src/services/firebase.ts` :
        - Web: Initialize Firebase App and Auth with default `browserLocalPersistence` .
        - Native: Initialize Firebase Auth using `initializeAuth(app, { persistence: getReactNativePersistence(AsyncStorage) })` via platform-switchable module.
    4. Implement `src/services/googleAuth.web.ts` :
        - Configure `GoogleAuthProvider` with `prompt: 'select_account'` and UX hint `hd: 'hbplus.fit'` .
        - Implement `signInWithPopup` with fallback to `signInWithRedirect` when popups are blocked.
    5. Implement `src/services/googleAuth.native.ts` :
        - Stub and integrate native Google Sign-In interface accepting web client ID and iOS client ID for EAS / bare React Native environments.
    6. Implement `src/services/authService.ts` exposing:
        - `signInWithGoogle()` , `signOut()` , `getCurrentUser()` , `subscribeToAuthChanges()` .
- **Relevant Files**:
    - `src/constants/config.ts` , `src/services/firebase.ts` , `src/services/authService.ts`
    - `src/services/googleAuth.web.ts` , `src/services/googleAuth.native.ts`
    - `src/types/auth.types.ts`
- **Dependencies**: Phase 1.
- **Security Considerations**: Never commit API keys or client secrets. Note that Firebase Web client config is public by design, but sensitive operations are gated by server-side rules.
- **Testing & Validation**: Unit test config loader; verify mock Firebase auth provider initialization.
- **Deliverables**: Functional Firebase auth service with platform abstraction.

Phase 3: Auth State Management & Domain Authorization Engine

- **Objective**: Implement the domain validation utility, error sanitization mapping, and the centralized React Context authentication state machine.
- **Specific Tasks**:
    1. Implement `src/utils/domainValidation.ts`:
        - Normalize email: `trim().toLowerCase()`.
        - Validate standard email format (exactly one `@` separator).
        - Exact domain match check against `hbplus.fit`.
        - Ensure `emailVerified === true`.
        - Explicitly reject invalid, missing, subdomain (`sub.hbplus.fit`), and spoofed (`hbplus.fit.example.com`) addresses.
    2. Implement `src/utils/errorMapper.ts`:
        - Map Firebase errors (`auth/popup-closed-by-user`, `auth/network-request-failed`, `DOMAIN_NOT_ALLOWED`) to sanitized application codes.
        - Enforce exact required PRD message for rejected domains: `"Access restricted. Please login using your official domain email."`
        - Prevent sensitive tokens or internal stack traces from leaking to the UI.
    3. Implement `src/context/AuthContext.tsx` implementing the 7-state finite state machine:
        - States: `initializing`, `unauthenticated`, `authenticating`, `authorizing`, `authorized`, `rejected`, `error`.
        - Immediate rejection sign-out: If domain check fails, immediately execute `signOut()` before setting state to `rejected` (SEC-07).
        - Re-entrancy guard to prevent double-invoking sign-in.
    4. Create `src/hooks/useAuth.ts` hook with runtime context validation.
- **Relevant Files**:
    - `src/utils/domainValidation.ts`, `src/utils/errorMapper.ts`
    - `src/context/AuthContext.tsx`, `src/hooks/useAuth.ts`
    - `src/types/auth.types.ts`
- **Dependencies**: Phase 2.
- **Security Considerations**: Immediate signout upon domain rejection ensures unauthorized users do not hold active sessions.
- **Testing & Validation**: Unit tests verifying the full domain validation test matrix (PRD Section 16.1).

- **Deliverables**: Robust AuthProvider state machine with automated unit test coverage.

Phase 4: Firestore User Profile Management & User Service

- **Objective**: Implement Firestore user synchronization for authorized users, capturing user profile metadata and audit timestamps.
- **Specific Tasks**:
    1. Define user profile schema in `src/types/user.types.ts`:
        - `uid`, `email`, `displayName`, `photoURL`, `role` (`'admin' | 'user'`), `createdAt`, `lastLoginAt`.
    2. Implement `src/services/userService.ts`:
        - `syncUserProfile(user: FirebaseUser): Promise<AppUserProfile>`
        - Reads existing `users/{uid}` document.
        - If document does not exist: creates profile with default role `'user'` (or `'admin'` if matching `EXPO_PUBLIC_INITIAL_ADMIN_EMAIL`), setting `createdAt: serverTimestamp()` and `lastLoginAt: serverTimestamp()`.
        - If document exists: updates `lastLoginAt: serverTimestamp()`, preserving existing `role`.
        - `getUserProfile(uid: string): Promise<AppUserProfile | null>`
        - `getAllUsers(): Promise<AppUserProfile[]>` (Admin only)
        - `updateUserRole(targetUid: string, newRole: 'admin' | 'user'): Promise<void>` (Admin only)
    3. Integrate profile sync into `AuthContext` authorization transition.
- **Relevant Files**:
    - `src/types/user.types.ts`, `src/services/userService.ts`, `src/context/AuthContext.tsx`
- **Dependencies**: Phase 3.
- **Security Considerations**: Firestore profiles can only be written if the client possesses a valid ID token from an `@hbplus.fit` account.
- **Testing & Validation**: Mock Firestore integration tests validating profile creation and timestamping.
- **Deliverables**: User profile management service linked to auth flow.

Phase 5: Role-Based Access Control (RBAC) & Secure Role Assignment

- **Objective**: Implement complete role-based authorization ensuring Admins have elevated capabilities while standard users are restricted.
- **Specific Tasks**:
    1. Extend `AuthContextValue` to provide `role: 'admin' | 'user' | null` and helper `isAdmin: boolean`.
    2. Implement admin seeding mechanism:
        - Configurable admin email list or environment setting.
        - On first login of an admin account, profile document assigns `role: 'admin'`.
    3. Implement RBAC authorization guards:
        - Screen-level authorization guards for `/admin` routes.
        - Service-level role validation before admin operations.
    4. Ensure client UI reflects role differences (Admin navigation items, management controls).
- **Relevant Files**:
    - `src/context/AuthContext.tsx`, `src/services/userService.ts`, `src/constants/config.ts`
- **Dependencies**: Phase 4.
- **Security Considerations**: Client-side role flag is strictly derived from the verified Firestore user document, which in turn is protected by Firestore Security Rules.
- **Testing & Validation**: Verify non-admin accounts cannot trigger `updateUserRole` or access admin views.
- **Deliverables**: End-to-end RBAC support in state and service layers.

Phase 6: Protected Navigation, Deep Linking & Screen Implementation

- **Objective**: Build the navigation system with URL deep linking and implement the core application screens: Login, User Dashboard, and Admin Dashboard.
- **Specific Tasks**:
    1. Install navigation packages: `@react-navigation/native`, `@react-navigation/native-stack`.
    2. Implement `src/navigation/linking.ts`:
        - Configure URL mapping: `login -> /login`, `dashboard -> /dashboard`, `admin -> /admin`.
    3. Implement `src/navigation/ProtectedRoute.tsx` & `src/navigation/AppNavigator.tsx`:

    - Conditional screen registration pattern: unauthenticated users only have `LoginScreen` in the stack; authorized users have `DashboardScreen`; admin users additionally have `AdminDashboardScreen`.
    - Show full-screen `LoadingIndicator` during `initializing` or `authorizing` states.
4. Implement `src/screens/LoginScreen.tsx`:
    - Responsive centered card layout (max-width 420px on desktop, fluid on mobile).
    - HB Plus branding, welcome headline, domain restriction notice.
    - `GoogleSignInButton` with loading indicator and disabled state during processing.
    - `ErrorMessage` banner with accessibility live region and retry button.
5. Implement `src/screens/DashboardScreen.tsx`:
    - Header with HB Plus branding, user profile picture/avatar fallback, display name, verified email, and role badge.
    - Welcome card and authentication status details.
    - Navigation link to Admin Dashboard if `isAdmin === true`.
    - Secure Logout button.
6. Implement `src/screens/AdminDashboardScreen.tsx`:
    - User management table/list showing registered `@hbplus.fit` users.
    - Display UID, display name, email, current role, and registration date.
    - Role toggle action (`user` <-> `admin`) invoking `userService.updateUserRole`.
    - Breadcrumb / back navigation to User Dashboard.

- **Relevant Files**:
    - `src/navigation/linking.ts`, `src/navigation/AppNavigator.tsx`, `src/navigation/ProtectedRoute.tsx`
    - `src/screens/LoginScreen.tsx`, `src/screens/DashboardScreen.tsx`, `src/screens/AdminDashboardScreen.tsx`
- **Dependencies**: Phase 3, Phase 5.
- **Security Considerations**: Unauthorized direct URL visits (e.g. browsing directly to `/dashboard` or `/admin`) immediately redirect to `/login` without exposing sensitive DOM or state.
- **Testing & Validation**: RNTL navigation tests testing deep links and route transitions.
- **Deliverables**: Complete navigation flow with Login, Dashboard, and Admin Dashboard screens.

Phase 7: Responsive UI Components, Cross-Platform Polish & Error Handling

- **Objective**: Create reusable presentation components, optimize touch targets and responsive behavior across web and mobile.
- **Specific Tasks**:
    1. Implement `src/components/GoogleSignInButton.tsx`:
        - Official Google styling guidelines (branding, logo, typography).
        - Disabled and loading spinner states.
        - Accessibility roles (`button`) and labels.
    2. Implement `src/components/LoadingIndicator.tsx`:
        - Fullscreen overlay for app initialization and inline spinner for button actions.
    3. Implement `src/components/ErrorMessage.tsx`:
        - Dismissible / retryable banner with accessible alert role.
    4. Implement `src/components/UserAvatar.tsx`:
        - Handles image load errors gracefully with fallback to user initials.
    5. Cross-platform testing & responsive verification:
        - Test responsive widths: Desktop (1200px+), Tablet (768px), Mobile (375px).
        - Ensure minimum 44x44pt interactive touch targets.
- **Relevant Files**:
    - `src/components/GoogleSignInButton.tsx`, `src/components/LoadingIndicator.tsx`, `src/components/ErrorMessage.tsx`, `src/components/UserAvatar.tsx`
    - `src/styles/theme.ts`
- **Dependencies**: Phase 6.
- **Security Considerations**: Error views must display user-friendly mapped strings, never raw exceptions or tokens.
- **Testing & Validation**: Responsive viewport test pass and component render tests.
- **Deliverables**: Polished, accessible, responsive UI component library.

Phase 8: Firestore Security Rules, Automated Rules Testing & Test Suite

- **Objective**: Enforce server-side security in Firestore, conduct automated rules unit testing, and establish comprehensive unit and integration tests.
- **Specific Tasks**:

1. Author `firebase/firestore.rules`:
    - Helper function `isOrgUser()` requiring `request.auth != null`, `request.auth.token.email_verified == true`, and email regex `^[^@]+@hbplus\.fit$`.
    - Helper function `isAdmin()` verifying admin role in the caller's user document.
    - Document rules for `/users/{uid}`:
        - Read: allowed if caller is admin OR caller is accessing own profile (`request.auth.uid == uid`).
        - Create: allowed if caller is org user creating own profile with matching email and non-admin role (or bootstrap admin).
        - Update: standard user can update own profile *excluding* `role`; admin can update any user's `role`.
        - Delete: denied.
    - Catch-all default deny: `match /{document=**} { allow read, write: if false; }`.
2. Create `firebase/firebase.json` for Firebase CLI and emulator suite configuration.
3. Implement unit and integration tests:
    - `__tests__/domainValidation.test.ts`: test full domain matrix.
    - `__tests__/errorMapper.test.ts`: test error translations.
    - `__tests__/AuthContext.test.tsx`: test state machine transitions and immediate sign-out on rejection.
    - `__tests__/firestore.rules.test.ts` (using `@firebase/rules-unit-testing`).

- **Relevant Files**:
    - `firebase/firestore.rules`, `firebase/firebase.json`
    - `__tests__/domainValidation.test.ts`, `__tests__/errorMapper.test.ts`, `__tests__/AuthContext.test.tsx`
- **Dependencies**: Phase 4, Phase 5.
- **Security Considerations**: Server rules strictly protect data regardless of client state manipulation.
- **Testing & Validation**: Execute `npm test` verifying 100% test pass rate across domain and state tests.
- **Deliverables**: Verified Firestore security rules and automated test suite.

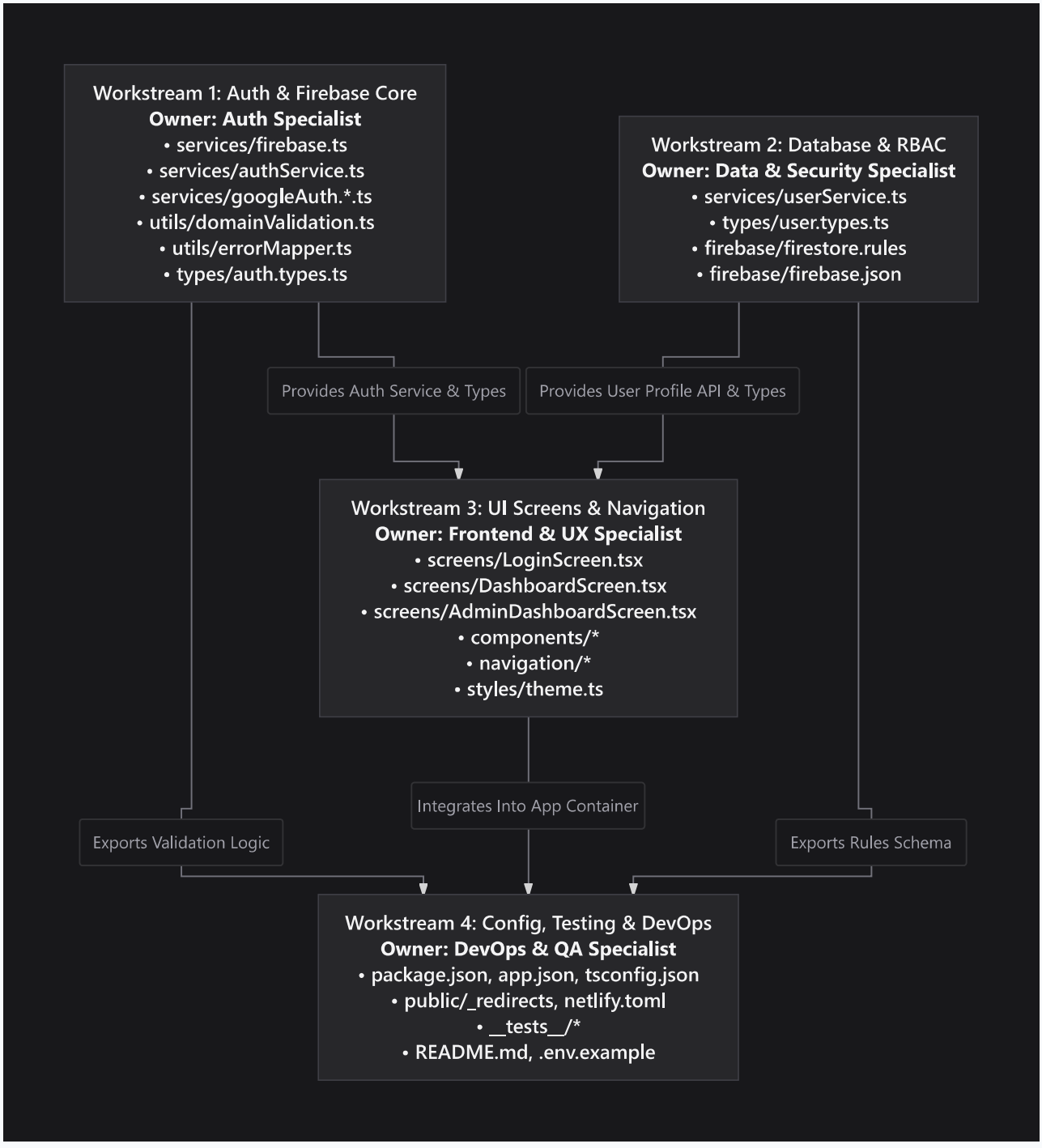
Phase 9: Production Web Deployment (Netlify), Mobile Build Config & Documentation

- **Objective**: Configure Netlify SPA deployment with HTTPS and security headers, configure mobile build descriptors, and produce comprehensive README and assignment deliverables.
- **Specific Tasks**:
    1. Create `public/_redirects` and `netlify.toml`:
        - SPA rewrite rule `/* /index.html 200` to prevent 404s on browser refresh or direct navigation to `/dashboard` or `/admin`.
        - Security headers: Content-Security-Policy, X-Content-Type-Options: nosniff, Referrer-Policy, Cross-Origin-Opener-Policy (`same-origin-allow-popups`).
    2. Configure web export build command: `npx expo export --platform web` outputting to `dist`.
    3. Configure mobile app descriptors in `app.json`:
        - Android package name, Google Services JSON configuration hook, permissions.
        - iOS bundle identifier, Google Services plist configuration hook, reversed client ID URL scheme.
    4. Write comprehensive `README.md` covering:
        - Architecture overview and technology stack.
        - Step-by-step local installation and startup instructions (Web, Android, iOS).
        - Environment variable configuration (`.env.example`).
        - Firebase project configuration & Google OAuth setup.
        - Domain restriction & RBAC logic explanation.
        - Testing guide (unit, integration, rules).
        - Production deployment guide & Netlify live URL.
        - Known limitations and security guarantees.
- **Relevant Files**:
    - `public/_redirects`, `netlify.toml`, `app.json`, `README.md`
- **Dependencies**: All prior phases.
- **Security Considerations**: Verify no `.env` or credential files are present in the build artifacts or repository.
- **Testing & Validation**: Verify production build bundle creation and validate Netlify rewrite config.

- **Deliverables**: Production-ready web export, deployment config, and comprehensive README.

Teamwork Breakdown for Parallel Execution

Following the `/teamwork` organizational model, the work is partitioned into **four independent, coordinated workstreams** with strict file ownership to prevent conflicts:



Workstream Responsibilities & File Ownership

Workstream	Focus Area	Exclusive File Ownership	Shared Interfaces / Contracts
Workstream 1: Auth & Firebase	Firebase initialization, Google OAuth adapters (web & native), domain validation, error mapping, AuthContext state machine	`src/services/firebase.ts` `src/services/authService.ts` `src/services/googleAuth.web.ts` `src/services/googleAuth.native.ts` `src/utils/domainValidation.ts` `src/utils/errorMapper.ts` `src/types/auth.types.ts` `src/context/AuthContext.tsx` `src/hooks/useAuth.ts`	Exports `useAuth()`, `AuthContextValue`, and `isAllowedDomain()` for UI and Data streams
Workstream 2: Database & RBAC	Firestore user profile synchronization, Admin user management APIs, role escalation rules, security rules	`src/services/userService.ts` `src/types/user.types.ts` `firebase/firestore.rules` `firebase/firebase.json`	Exports `AppUserProfile`, `userService.syncUserProfile`, `getAllUsers`, and `updateUserRole`
Workstream 3: UI & Navigation	Login screen, User Dashboard, Admin Dashboard, Google Sign-In button, avatars, loading/error widgets, React Navigation & route guards	`src/screens/LoginScreen.tsx` `src/screens/DashboardScreen.tsx` `src/screens/AdminDashboardScreen.tsx` `src/components/*` `src/navigation/*` `src/styles/theme.ts`	Consumes `useAuth()` and `userService`; owns all visual rendering and navigation links

Workstream	Focus Area	Exclusive File Ownership	Shared Interfaces / Contracts
Workstream 4: DevOps & QA	Expo build configuration, Netlify configuration, environment validation, unit/integration test suites, README documentation	`package.json` `app.json` `tsconfig.json` `metro.config.js` `public/_redirects` `netlify.toml` `.env.example` `README.md` `__tests__/*`	Verifies integration points, runs build scripts and automated test suites

Coordination & Conflict Prevention Protocols

1. **Shared Types First**: `src/types/auth.types.ts` and `src/types/user.types.ts` are established upfront before implementation starts.
2. **Mocking at Boundaries**: Frontend UI workstream develops against typed interfaces using mock user fixtures while Auth and Database streams implement the underlying Firebase calls.
3. **Single Integration Point**: `App.tsx` and `AuthContext.tsx` serve as the coordinated integration boundaries.

Verification Plan

Automated Tests

1. **Domain Validation Test Suite**:

```
bash
npm test -- __tests__/domainValidation.test.ts
```

   - Validates `@hbplus.fit` allowed.
   - Validates `@gmail.com`, `@yahoo.com`, `@anothercompany.com` rejected.
   - Validates subdomain `@sub.hbplus.fit` rejected.
   - Validates spoofed `@hbplus.fit.example.com` rejected.
   - Validates email verification requirement (`emailVerified === true`).

2. **Error Mapper Test Suite**:

```
bash
npm test -- __tests__/errorMapper.test.ts
```

- Validates exact error message: `"Access restricted. Please login using your official domain email."`
- Validates cancelled auth returns silent state.

3. **Auth State Machine Test Suite**:

```
bash
npm test -- __tests__/AuthContext.test.tsx
```

- Validates state transitions (`initializing` -> `unauthenticated` -> `authenticating` -> `authorizing` -> `authorized` or `rejected`).
- Validates immediate `signOut()` invocation on domain rejection.

4. **TypeScript & Build Verification**:

```
bash
npx tsc --noEmit
npx expo export --platform web
```

Manual Verification

1. **Web Browser Google Sign-In**:
   - Sign in with an `@hbplus.fit` account: redirects to `/dashboard`, displays user profile and role.
   - Sign in with a standard `@gmail.com` account: immediately signs out, remains on `/login`, displays exact PRD restriction banner.
   - Refresh browser on `/dashboard`: session persists without redirect flicker.
   - Click Logout: signs out from Firebase, clears session, redirects to `/login`.
2. **RBAC & Admin Management**:
   - Log in with Admin user: navigation displays Admin link; clicking leads to `/admin`.
   - Admin view lists registered users and allows role toggle.
   - Log in with standard user: direct URL navigation to `/admin` redirects to `/dashboard` or `/login`.
3. **Netlify SPA Direct Routing**:

- Directly load `https://<site>.netlify.app/dashboard`: does not return a 404 error; correctly displays login screen or dashboard depending on auth session.